

Python
for DA

Polars: сложные фильтры и группировка



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

-  Камера должна быть включена на протяжении всего занятия
-  В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
-  Вести себя уважительно и этично по отношению к остальным участникам занятия
-  Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
-  Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

Повторение

-  Обзор библиотеки Polars и его ключевые особенности
-  Мотивация использования Polars и установка библиотеки polars
-  DataFrame в Polars
-  Загрузка данных в Polars DataFrame
-  Создание DataFrame в Polars
-  Контексты и выражения Polars
-  Выбор и фильтрация данных в Polars
-  Трансформация данных

План занятия

- Обработка пропусков
- Изменение типов данных. Кастинг
- Операции агрегации
- Повторение describe для одного поля
- Groupby
- Оконные функции
- Уникальные значения
- Сложная фильтрация
- Условия



ОСНОВНОЙ БЛОК





Сложные фильтры и группировка

Пример

Скачаем данные по [ссылке](#)

```
import polars as pl
data=pl.read_csv('books_data.csv', null_values='NaN')
print(data.head())
print(data.describe())
```




```
pl.read_csv('books_data.csv', null_values='NaN')
```

Считывает CSV-файл в Polars DataFrame, указывая 'NaN' в качестве представления для нулевых значений



ВОПРОСЫ





Обработка пропусков





Обработка пропусков

Каждый столбец в DataFrame (или эквивалентно Series) представляет собой массив Arrow или набор массивов Arrow, основанный на формате Apache Arrow. Отсутствующие данные представляются в Arrow и Polars с помощью нулевого значения. Нулевое пропущенное значение применяется для всех типов данных, включая числовые значения.



NotaNan или NaN

Polars также допускает значения NotaNan или NaN для столбцов с плавающей запятой. Эти значения NaN рассматриваются как тип данных с плавающей точкой, а не как пропущенные данные. Вы можете вручную определить пропущенное значение с помощью функции python None.



Определение недостающих данных

Прежде чем работать с отсутствующими данными, необходимо определить, где в вашем DataFrame имеются отсутствующие значения. Вы можете использовать такие функции, как `is_null()` или `describe()`, чтобы получить сводку отсутствующих значений в каждом столбце.

Удаление пропущенных данных

```
df = df.drop_nulls() # Отбрасываем строки с любыми пропущенными значениями
```

Заполнение пропущенных данных

```
df = df.fill_null(0) # Замена отсутствующих значений на 0
```


Прямое и обратное заполнение

```
df.with_columns(pl.col('has_nulls').forward_fill()) # Прямое заполнение  
недостающих значений в определенном столбце
```

Интерполяция

```
df = df.interpolate() # Линейная интерполяция для недостающих значений в  
определенном столбце
```

Пример

```
data = data.with_columns(pl.col('year').forward_fill())  
data.describe()
```



ВОПРОСЫ





Изменение типов данных. Кастинг



Кастинг

Преобразует базовый DataType столбца в новый. Polars использует Arrow для управления данными в памяти и полагается на вычислительные ядра в реализации Rust для выполнения преобразования. Доступен с помощью метода `cast()`



cast

Это метод, который включает параметр `strict`, определяющий поведение Polars при встрече со значением, которое не может быть преобразовано из исходного `DataType` в целевой `DataType`.

Пример

```
data = data.with_columns(  
    data['year'].cast(pl.Int32) # or pl.Int64 if larger values are expected  
)  
data.head(5)
```




ВОПРОСЫ





Операции агрегации



quantile()

Вычисляет значения квантилей для числовых данных в DataFrame.



mean()

Вычисляет среднее (усредненное) значение числовых данных в каждой группе. Это мера центральной тенденции, которая позволяет понять типичное значение в каждой группе.



`sum()`

Вычисляет сумму числовых данных в каждой группе.



`min()` и `max()`

Вычисляют минимальное и максимальное значения соответственно в каждой группе.



count()

Подсчитывает количество ненулевых значений в каждой группе.



`std()` и `var()`

Вычисляют стандартное отклонение и дисперсию, соответственно, в каждой группе.



Медиана

Это среднее значение в отсортированном списке числовых данных, устойчивое к промахам.



ВОПРОСЫ





**Повторение describe
для одного поля**

Столбец describe()

```
df.select([
    pl.col("price").count().alias("count"),
    pl.col("price").null_count().alias("null_count"),
    pl.col("price").mean().alias("mean"),
    pl.col("price").std().alias("std_dev"),
    pl.col("price").median().alias("median"),
    pl.col("price").min().alias("min"),
    pl.col("price").quantile(0.25).alias("25%"),
    pl.col("price").quantile(0.5).alias("50%"),
    pl.col("price").quantile(0.75).alias("75%"),
    pl.col("price").max().alias("max"),
])
```



```
pl.col("price").count().alias("count")
```

Эта операция подсчитывает количество ненулевых значений в столбце "price" и присваивает результат новому столбцу с именем "count".



```
pl.col("price").null_count().alias("null_count")
```

Эта операция вычисляет количество нулевых (отсутствующих) значений в столбце "price" и присваивает результат новому столбцу с именем "null_count".



```
pl.col("price").mean().alias(  
"mean")
```

Эта операция вычисляет среднее значение столбца "price" и присваивает результат новому столбцу с именем "mean".



```
pl.col("price").std().alias("std_dev")
```

Эта операция вычисляет стандартное отклонение значений в столбце "price" и присваивает результат новому столбцу с именем "std_dev".



```
pl.col("price").median(  
).alias("median")
```

Эта операция вычисляет медианное значение столбца "price" и присваивает результат новому столбцу с именем "median".



```
pl.col("price").min().alias("min")
```

Эта операция находит минимальное значение в столбце "price" и присваивает результат новому столбцу с именем "min".



```
pl.col("price").quantile(0.25).  
alias("25%")
```

Эта операция вычисляет 25-й процентиль (первый квартиль) значений в столбце "price" и присваивает результат новому столбцу с именем "25%".



```
pl.col("price").quantile(0.5)  
.alias("50%")
```

Эта операция вычисляет 50-й процентиль (медиану) значений в столбце "price" и присваивает результат новому столбцу с именем "50 %".



```
pl.col("price").quantile(  
0.75).alias("75%")
```

Эта операция вычисляет 75-й процентиль (третий квартиль) значений в столбце "price" и присваивает результат новому столбцу с именем "75 %".



```
pl.col("price").max().alias("max")
```

Эта операция находит максимальное значение в столбце "price" и присваивает результат новому столбцу с именем "max".



ВОПРОСЫ





Groupby





groupby

Это операция для группировки данных на основе одного или нескольких столбцов и выполнения агрегированных операций над этими группами



groupby()

Это функция, которая используется для группировки данных по одному или нескольким столбцам, создавая группы на основе уникальных значений в этих столбцах.

Синтаксис

```
DataFrame.groupby([col1, col2, ...])
```



agg()

Это функция, которая используется для указания операций агрегирования, которые должны быть выполнены над каждой группой. Она принимает словарь, в котором ключами являются имена столбцов, а значениями - применяемые функции агрегирования.

Синтаксис

```
GroupBy.agg({'col1': 'agg_func1', 'col2': 'agg_func2', ...})
```



first() и last()

Возвращает первое или последнее значение в каждой группе.



`pivot()`

Эта функция изменяет форму данных на основе значений указанных столбцов, создавая разворотную таблицу. Применяется для создания перекрестных таблиц и обобщения данных.

Пример

```
data.groupby("type").agg(
    [
        pl.mean("price").alias("mean_price"),
        pl.median("year").alias("median_year"),
        pl.count(),
    ]
)
```


Пример

```
data.groupby(["type", "year"]).agg([
    pl.count()
]).pivot(
    values="count",
    index=["year"],
    columns=['type'])
```



ВОПРОСЫ





Оконные функции



Оконные функции

Это функции, которые позволяют выполнять вычисления в определенном окне строк в DataFrame, обеспечивают гибкий способ вычисления агрегатов, ранжирования и других операций в определенном окне, таких как скользящее среднее или кумулятивная сумма.



over()

Это функция, которая используется для указания окна, над которым работает оконная функция. Она определяет границы окна на основе критериев разбиения и упорядочивания.

Over

СИНТАКСИС

```
df.with_columns( pl.col("target_column").agg_function().over("group_column") )
```

ПОЯСНЕНИЕ

`pl.col("target_column")` – столбец, к которому применяется агрегационная функция.

`agg_function()` – агрегационная функция (min, max, sum, mean и т.д.).

`over("group_column")` – столбец (или список столбцов), по которому создается окно.

Over

пример

```
df.with_columns([  
    pl.col("price").mean().over("type").alias("mean_price_by_type")  
]).head(10)
```



ВОПРОСЫ





Уникальные значения

Способы подсчета уникальных значений

Точная методика

Аппроксимация

Пример

```
data.select(
  pl.col("year").n_unique().alias("unique"),
  pl.approx_n_unique("year").alias("unique_approx"),
)
```



ВОПРОСЫ





Сложная фильтрация



Сложные фильтры

Позволяют применять расширенные условия фильтрации к фрейму данных, позволяя выбирать определенные строки на основе сложных критериев. Могут включать в себя несколько условий, логические операторы и даже пользовательские функции.



Условия фильтрации

Это выражения, которые оцениваются как True или False для каждого ряда в DataFrame. Они могут включать в себя сравнения, логические операторы и функции.



Логические операторы

& (AND), | (OR) и ~ (NOT) - операторы, которые можно использовать для объединения нескольких условий.



Операторы сравнения

`==` (равно), `!=` (не равно), `<` (меньше), `>` (больше), `<=` (меньше или равно) и `>=` (больше или равно), используются для сравнения значений.

Пример

```
df.filter((pl.col("возраст") > 30) & (pl.col("пол") == "Мужской"))
```

```
df.filter(~(pl.col("category") == "High"))
```



Функции

Для выполнения сложных вычислений или преобразований можно использовать встроенные функции или функции, определяемые пользователем в условиях фильтрации.

Пример

```
def is_adult(age):  
    return age >= 18  
  
df.filter(is_adult(pl.col("age")))
```



Цепочка фильтров

Несколько условий фильтрации можно объединить в цепочку, чтобы создать более сложные критерии фильтрации.

Синтаксис

```
DataFrame.filter(condition1).filter(condition2)
```



Комбинирование фильтров

Фильтры можно комбинировать с другими операциями DataFrame, такими как `select()`, `groupby()` и `agg()`, для выполнения сложных манипуляций с данными.



ВОПРОСЫ





Условия





Условия

Условия типа if-else в выражениях с синтаксисом when, then, otherwise. Предикат помещается в выражение when, и когда он оценивается как true, применяется выражение then, в противном случае применяется выражение otherwise (по порядку).

Пример

```
data.select(  
  pl.col("year"),  
  pl.when(pl.col("year") > 2014)  
    .then(pl.lit(True))  
    .otherwise(pl.lit(False))  
    .alias("conditional"),  
)
```



ВОПРОСЫ





ПРАКТИЧЕСКАЯ РАБОТА



Задание

- Предположим, у Вас есть данные о типах зданий, их метраже и годе постройки. Как на основе этих данных построить такую таблицу?

building_type	mean_sqft	median_year	count
---	---	---	---
str	f64	f64	u32
C	999.854722	2009.0	1692
A	989.539918	2009.0	1653
B	992.754444	2009.0	1655

- Что выведет следующий код?

```
data.with_columns([
    pl.col("price").sum().over("type").alias("sum_price_by_type")
]).head(10)
```



ВОПРОСЫ



Домашнее задание

1. Загрузите данные на свой жёсткий диск.
2. Считайте их в polars.
3. Посчитайте уникальное количество кодов категорий двумя разными способами.
4. Замените отсутствующие коды категорий значением по выбору и убедитесь, что описательная статистика поменялась.
5. На основе оконных функций постройте поле, состоящее из максимальной цены в разбивке по коду категории.
6. Постройте новое поле, принимающее значение True, если цена больше средней цены, и False в ином случае.

Заключение

